In [5]:
```
import gzip
import networkx as nx
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict
import random
```

In [6]:
```
# Set seed for reproducibility
random.seed(42)
np.random.seed(42)
```

In [7]:
```
file_path="/content/drive/MyDrive/Stage7_Orkut_Relationship/Copy of com-orkut.ungra
df = pd.read_csv(file_path, sep="\t", comment="#", header=None, names=["users", "co
```

In [8]:
```
df.head()
```

Out[8]:

	users	connections
0	1	2
1	1	3
2	1	4
3	1	5
4	1	6

In [9]:
```
df.shape
```

Out[9]:
```
(117185083, 2)
```

In [10]:
```
sample_data = df.head(2000000).values.tolist()
```

In [11]:
```
G = nx.Graph()
G.add_edges_from(sample_data)
print(f"Number of nodes: {G.number_of_nodes()}")
print(f"Number of edges: {G.number_of_edges()}")
```

```
Number of nodes: 646240
Number of edges: 2000000
```

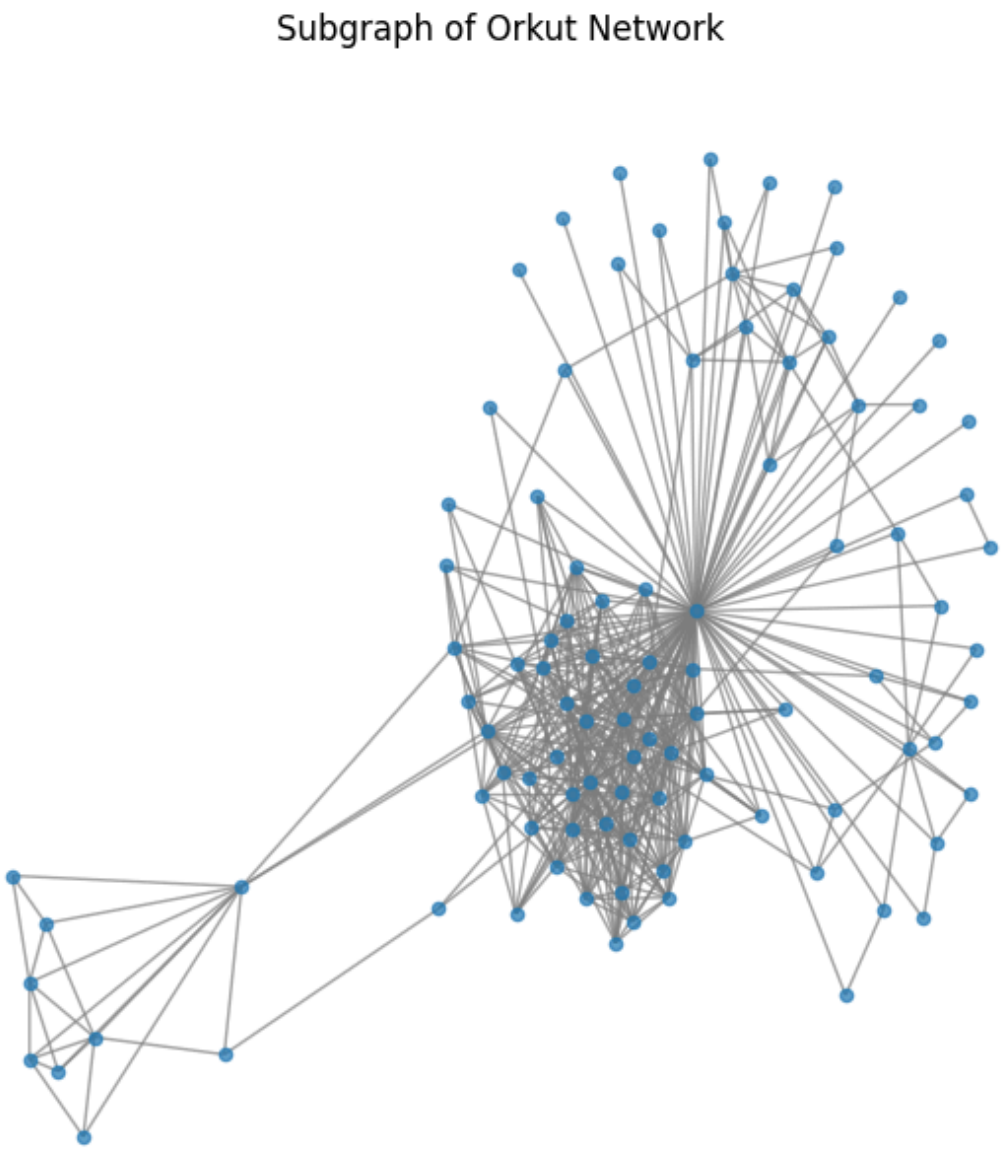
In [12]:
```
# Check if the network is connected
print(f"Is the Graph Connected? {nx.is_connected(G)}")
```

```
Is the Graph Connected? True
```

The code below show the subgraph for the first 100 nodes

In [13]:
```
# Extract a small subgraph (first 100 nodes)
small_nodes = list(df["users"].unique())[:100]
small_G = G.subgraph(small_nodes)
```

```
# Plot the small network
plt.figure(figsize=(6, 6))
nx.draw(small_G, with_labels=False, node_size=20, edge_color="gray", alpha=0.7)
plt.title("Subgraph of Orkut Network")
plt.show()
```



Loading the 5000 ground truth communities dataset

In [14]:
```
import pandas as pd
import gzip

file_path="/content/drive/MyDrive/Stage7_Orkut_Relationship/Copy of com-orkut.top50

communities = []
with gzip.open(file_path, 'rt', encoding='utf-8') as f:
    for line in f:
        line = line.strip()
```

```
        if line.startswith('#') or not line:
            continue
        members = set(map(int, line.split('\t')))

        communities.append(members)

df = pd.DataFrame({'community_members': [list(c) for c in communities]})
```

In [15]:
```
# Check if the network is connected
print(f"Is the Graph Connected? {nx.is_connected(G)}")
```

```
Is the Graph Connected? True
```

In [16]:
```
df.head()
```

Out[16]:

	community_members
0	[2, 5, 6, 49157, 974856, 466953, 270346, 22160...
1	[2, 319490, 77828, 114692, 319492, 319495, 319...
2	[45061, 628742, 696325, 706565, 671757, 706573...
3	[1539, 516, 1541, 6, 4616, 527, 18, 19, 153620...
4	[8, 771627, 26646, 26648, 26649, 772381, 77187...

In [17]:
```
df.shape
```

Out[17]:
```
(5000, 1)
```

To check the total unique users in the top 5000 communities and make every user in the communities is aleast a node in Graph

In [18]:
```
# Get all nodes in these communities
all_nodes = set()
for comm in communities:
    all_nodes.update(comm)
print(f"Total unique users in top 5,000 communities: {len(all_nodes):,}")
```

```
Total unique users in top 5,000 communities: 731,514
```

In [19]:
```
# Ensure isolated community nodes are in G
for node in all_nodes:
    if node not in G:
        G.add_node(node)

print(f"Graph: {G.number_of_nodes()} nodes, {G.number_of_edges()} edges")
```

```
Graph: 1065059 nodes, 2000000 edges
```

Implementing the four structural metrics of a good community

In [20]:
```
# Metric Functions
def density(G, nodes):
```

```
    n = len(nodes)
    if n < 2: return 0
    e = G.subgraph(nodes).number_of_edges()
    return e / (n * (n - 1) / 2)
```

In [21]:
```
def separability(G, nodes):
    nodes = set(nodes)
    internal = external = 0
    for u in nodes:
        for v in G.neighbors(u):
            if v in nodes:
                internal += 1
            else:
                external += 1
    internal //= 2
    return internal / external if external > 0 else 1e6
```

In [23]:
```
def triad_ratio(G, nodes):
    if len(nodes) < 3: return 0
    subG = G.subgraph(nodes)
    tri = nx.triangles(subG)
    return sum(1 for u in nodes if tri[u] > 0) / len(nodes)
```

In [24]:
```
def is_connected(G, nodes):
    if len(nodes) <= 1: return True
    return nx.is_connected(G.subgraph(nodes))
```

Calculating the above structural metrics for all the top 5000 ground truth communities

In [25]:
```
#  Compute metrics for ground-truth communities

gt_data = []
for comm in communities:
    gt_data.append({
        'size': len(comm),
        'density': density(G, comm),
        'separability': separability(G, comm),
        'clustering': triad_ratio(G, comm),
        'cohesive': is_connected(G, comm)
    })

gt_df = pd.DataFrame(gt_data)
print("Metrics computed for all 5,000 communities.")
print("\nSummary Statistics:")
print(gt_df[['density', 'separability', 'clustering', 'cohesive']].describe())
```

```
Metrics computed for all 5,000 communities.

Summary Statistics:
           density    separability   clustering
count  5000.000000     5000.000000  5000.000000
mean      0.006684    93600.049628     0.056056
std       0.035861   291300.535376     0.166761
min       0.000000        0.000000     0.000000
25%       0.000000        0.000000     0.000000
50%       0.000000        0.000000     0.000000
75%       0.001514        0.123915     0.000000
max       0.794872  1000000.000000     1.000000
```

Creating a baseline communities to created random groups of the same size and compare it with the real communities

In [26]:
```
# Random baseline (sample 100 communities)

rand_data = []
for comm in communities[:100]:
    n = len(comm)
    d_list = []
    s_list = []
    c_list = []
    for _ in range(50):
        rand_nodes = random.sample(list(G.nodes()), n)
        d_list.append(density(G, rand_nodes))
        s_list.append(separability(G, rand_nodes))
        c_list.append(triad_ratio(G, rand_nodes))
    rand_data.append({
        'density': np.mean(d_list),
        'separability': np.mean(s_list),
        'clustering': np.mean(c_list)
    })
rand_df = pd.DataFrame(rand_data)

print("Random baseline computed.")
print(f"Real vs Random Density: {gt_df['density'][:100].mean():.4f} vs {rand_df['de
print(f"Real vs Random Clustering: {gt_df['clustering'][:100].mean():.4f} vs {rand_
```

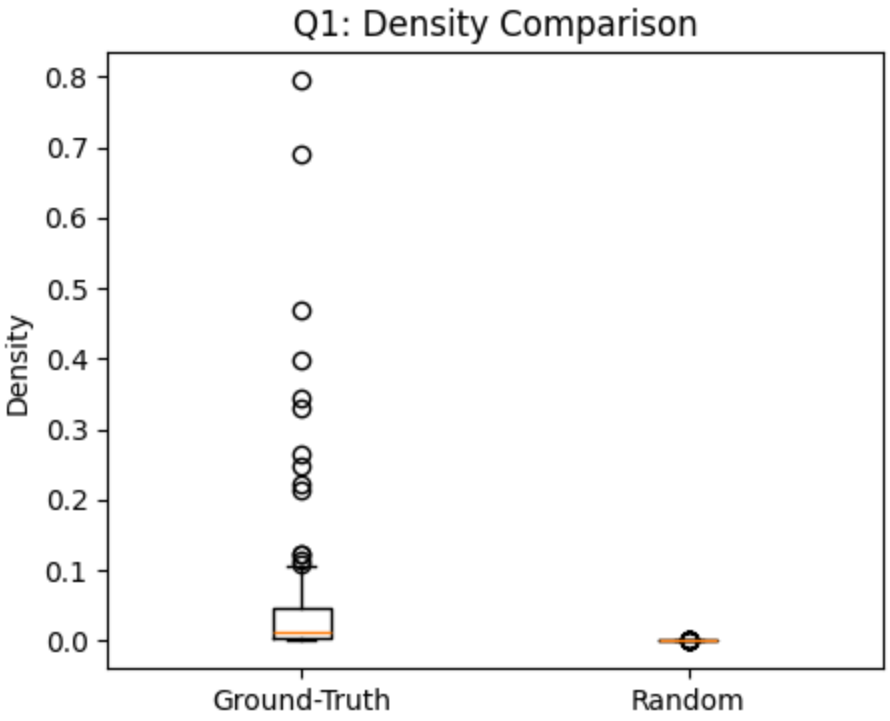
```
Random baseline computed.
Real vs Random Density: 0.0592 vs 0.0000
Real vs Random Clustering: 0.4774 vs 0.0000
```

Answering the research questions

In [27]:
```
# --- Answers to Research Questions ---

# Q1: How dense are real Orkut communities compared to random node sets
gt_d = gt_df['density'].mean()
rand_d = rand_df['density'].mean()
ratio = gt_d / rand_d if rand_d != 0 else float('inf')
print(f"Q1: Density = {gt_d:.8f} (GT) vs {rand_d:.8f} (Random) → {gt_d/rand_d:.0f}×
```

```
Q1: Density = 0.00668394 (GT) vs 0.00000342 (Random) → 1953× denser
```

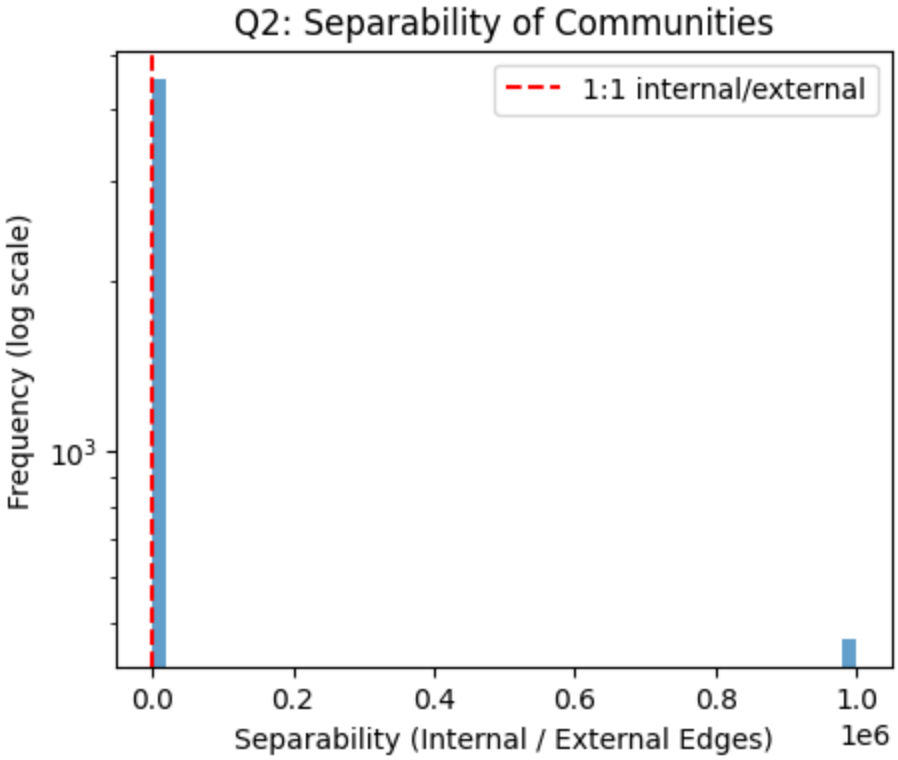
In [28]:
```
plt.figure(figsize=(5,4))
plt.boxplot([gt_df['density'][:100], rand_df['density']], tick_labels=['Ground-Trut
plt.ylabel('Density')
plt.title('Q1: Density Comparison')
plt.savefig('q1_density.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [29]:
```
# Q2: Are ground-truth communities well-separated from the rest of the network

rand_df = pd.DataFrame(rand_data)
med_sep = gt_df['separability'].median()
pct_low = (gt_df['separability'] < 1).mean() * 100
print(f"Q2: Median separability = {med_sep:.5f}; {pct_low:.2f}% have more external
```

```
Q2: Median separability = 0.00000; 90.62% have more external than internal edges
```

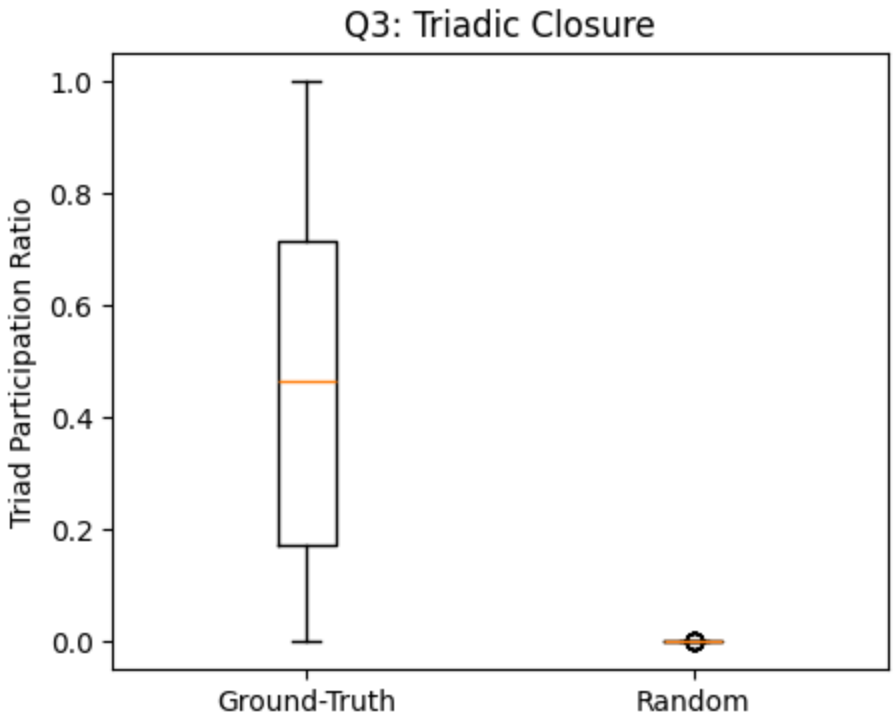
In [30]:
```
plt.figure(figsize=(5,4))
plt.hist(gt_df['separability'], bins=50, log=True, alpha=0.7)
plt.axvline(1, color='red', linestyle='--', label='1:1 internal/external')
plt.xlabel('Separability (Internal / External Edges)')
plt.ylabel('Frequency (log scale)')
plt.title('Q2: Separability of Communities')
plt.legend()
plt.savefig('q2_separability.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [31]:
```
# Q3: Do Orkut groups exhibit strong triadic closure (friends of friends are friend
gt_c = gt_df['clustering'].mean()
rand_c = rand_df['clustering'].mean()
print(f"Q3: Clustering = {gt_c:.4f} (GT) vs {rand_c:.4f} (Random) → {gt_c/rand_c:.0
```

```
Q3: Clustering = 0.0561 (GT) vs 0.0000 (Random) → 4221× stronger
```

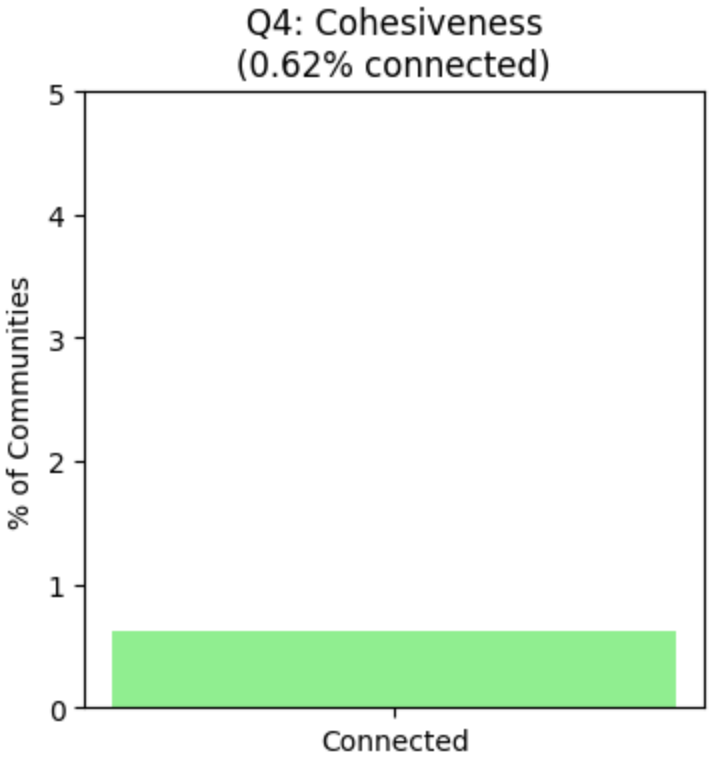
In [32]:
```
plt.figure(figsize=(5,4))
plt.boxplot([gt_df['clustering'][:100], rand_df['clustering']], tick_labels=['Groun
plt.ylabel('Triad Participation Ratio')
plt.title('Q3: Triadic Closure')
plt.savefig('q3_clustering.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [33]:
```
# Q4: How cohesive are these communities, are they connected subgraphs
coh_pct = gt_df['cohesive'].mean() * 100
print(f"Q4: {coh_pct:.2f}% of communities are connected subgraphs")
```

```
Q4: 0.62% of communities are connected subgraphs
```

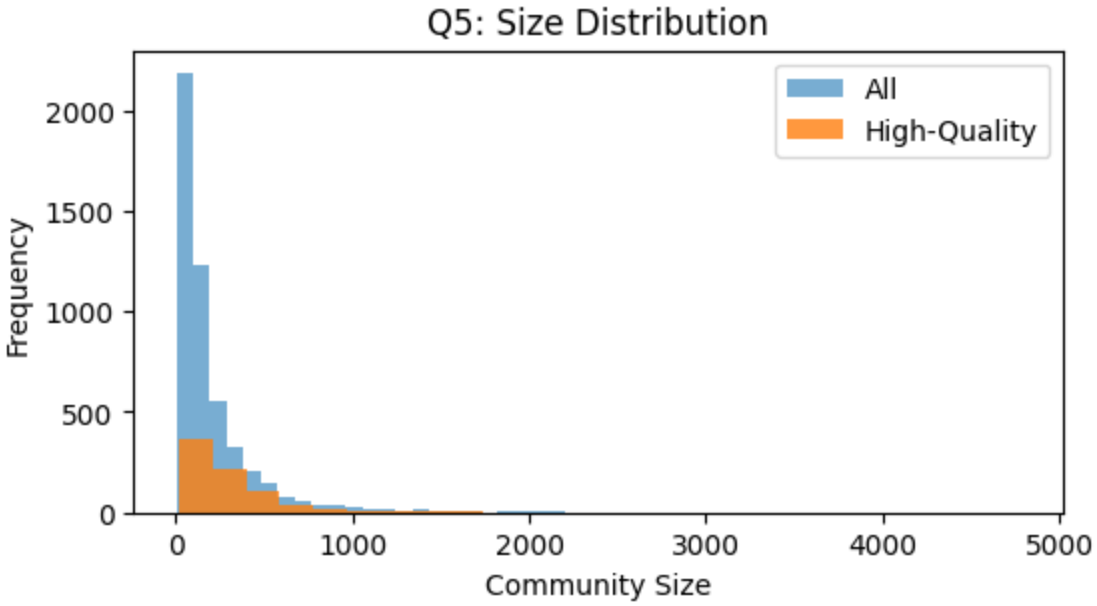
In [34]:
```
plt.figure(figsize=(4,4))
plt.bar(['Connected'], [coh_pct], color='lightgreen')
plt.ylim(0, 5)
plt.ylabel('% of Communities')
plt.title(f'Q4: Cohesiveness\n({coh_pct:.2f}% connected)')
plt.savefig('q4_cohesiveness.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [35]:
```
# Q5: What is the size distribution of high-quality communities
# Define high-quality: top 25% on both density and clustering
gt_df['high_quality'] = (
    (gt_df['density'] > gt_df['density'].quantile(0.75)) &
    (gt_df['clustering'] > gt_df['clustering'].quantile(0.75))
)
median_size = gt_df[gt_df['high_quality']]['size'].median()
print(f"Q5: Median size of high-quality communities = {median_size:.0f} users")
```

```
Q5: Median size of high-quality communities = 223 users
```

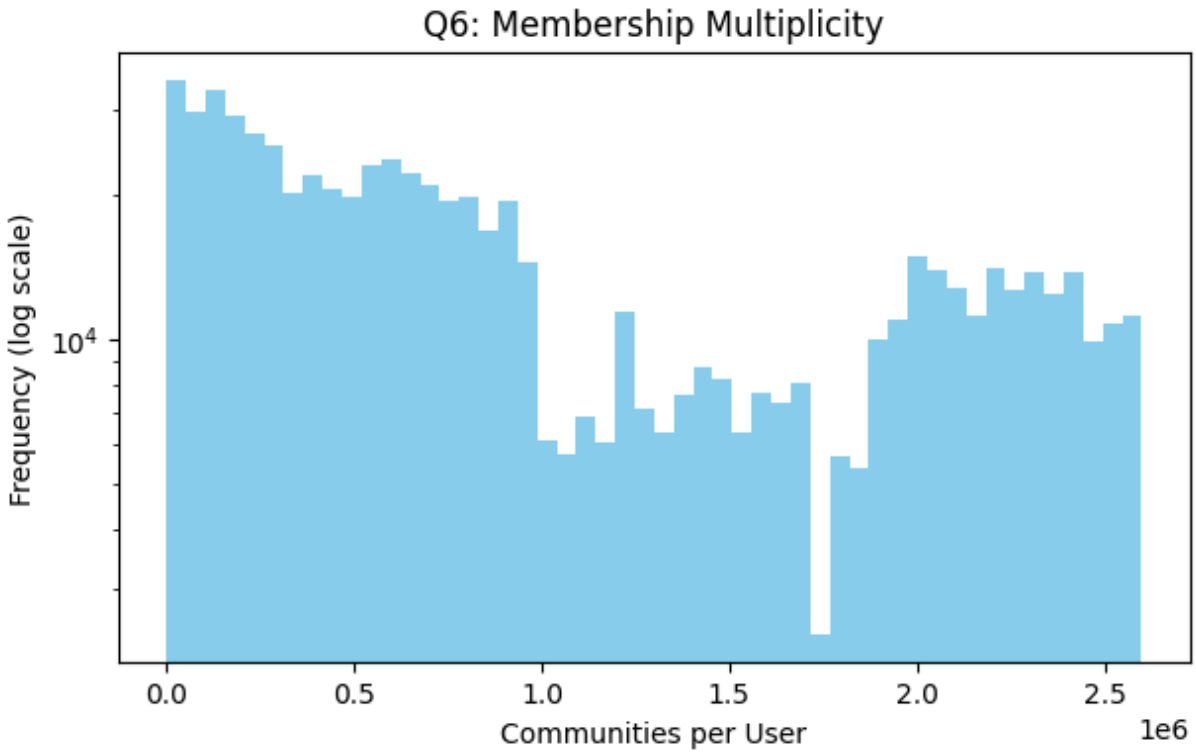
In [40]:
```
plt.figure(figsize=(6,3))
plt.hist(gt_df['size'], bins=50, alpha=0.6, label='All')
plt.hist(gt_df[gt_df['high_quality']]['size'], bins=25, alpha=0.8, label='High-Qual
plt.xlabel('Community Size')
plt.ylabel('Frequency')
plt.title('Q5: Size Distribution')
plt.legend()
plt.savefig('q5_size.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [41]:
```
# Q6: How many ground-truth communities does a typical user belong to in Orkut,
# how does this membership multiplicity relate to their connectivity (degree) in th
from collections import defaultdict
import gzip

user_memberships = defaultdict(int)

with gzip.open('/content/drive/MyDrive/Stage7_Orkut_Relationship/Copy of com-orkut.
    for line in f:
        users = map(int, line.strip().split())
        for u in users:
            user_memberships[u] += 1

memberships = list(user_memberships.values())
print(f"Average memberships per user: {sum(memberships)/len(memberships):.1f}")
print(f"Median: {sorted(memberships)[len(memberships)//2]}")
```

```
Average memberships per user: 1.5
Median: 1
```

In [42]:
```
import matplotlib.pyplot as plt
plt.figure(figsize=(7,4))
plt.hist(user_memberships, bins=50, log=True, color='skyblue')
plt.xlabel('Communities per User')
plt.ylabel('Frequency (log scale)')
plt.title('Q6: Membership Multiplicity')
plt.savefig('q6_membership.png', dpi=150, bbox_inches='tight')
plt.show()
```


Q6: Membership Multiplicity
Frequency (log scale)
10^4
0.0
0.5
1.0
1.5
2.0
2.5
Communities per User
1e6